History Before Models

Labelled history, small language models and cascades for request classification on local hardware. A measured study

Giovanni Simonetti (ORCID 0009-0008-7543-6737)

Technical report, draft 0.2, October 2026

Abstract

Organisations that want to classify incoming requests with language models usually start by choosing a model. This report measures what happens if they start from the requests they have already classified instead. On a public workload of 4,500 crowdsourced requests routed to 10 departments, with 15,000 labelled past requests, we run four open-weight models of one family (0.5 to 7.6 billion parameters) locally on an office PC with no graphics card, and compare them with two ways of using the history: a deterministic lookup and a standard text classifier. Three results stand out. First, the classifier trained on the history answers 95.9% of requests correctly (95% interval 95.1% to 96.7%), against 86.6% for the largest model alone, at a cost too small to compare. Second, cascades of models only, small first and large when unsure, cost 15% to 74% more than the large model alone when accuracy must stay within half a point of it, in large part because the small models' confidence does not separate their right answers from their wrong ones well enough. Third, the language model earns its place where the history is thin: with fewer than about 100 labelled examples per category the 7.6B model alone beats the classifier, while a classifier that hands its uncertain requests to the model is at least as accurate as the model alone at every size of history, sends it 57% of requests or fewer from 32 examples per category, and is clearly more accurate from 64 (+2.7 points, interval +1.9 to +3.4). The order that follows is to capture the history first, use it with the simplest tool that works, and keep the language model for what the history does not cover. It is one task, one model family and one machine, and we state what would count against the result. Code, stored model outputs and every number are public.

1. Introduction

Classifying incoming requests is one of the most common uses of language models in services: a ticket, an email or a chat message arrives and must reach the right team. The usual question is which model to use, and how to spend less on it. Model cascades answer the second part: a small model answers first and passes the request to a larger one only when its calibrated confidence is too low [1, 2, 3].

Most organisations that classify requests, however, already hold something more valuable than any model: thousands of past requests with the category a person gave them. This report asks what that history is worth, measured on real models running on ordinary hardware, and in what order history and models should be used.

An earlier draft of this work [15] studied in simulation when a cascade should have a third stage, and proposed a rule of thumb: when neighbouring models are less than 5x apart in cost, the third stage should be rules, not a middle model. Running real models changed the question. A stage built on the labelled history did more than any model, and a plain classifier trained on the same history did more still. This report presents those measurements.

The contributions are:

- **A measured comparison** of four local models, cascades of them, a deterministic lookup in the history and a text classifier trained on it, on the same requests, split and cost measure.
- **A curve of how much history is needed** before the classifier overtakes a language model that sees no examples, and of what a classifier that hands uncertain requests to the model achieves along the way.

- **A first real-model check of the 5x rule** of [15], within the range of costs the test covers.
- **Everything needed to repeat it**: code, the stored answers of every model, and the protocol for running it on other models and workloads.

None of the components is new. Cascades, calibrated confidence, nearest-neighbour lookup and linear text classifiers are all established. What the report adds is the measurement, on hardware a small organisation owns, and the order it suggests.

2. Related work

Cascades and routing. FrugalGPT [1] chains model APIs and stops at the first acceptable answer; AutoMix [7] uses self-verification to decide when to escalate. UCCI [2] calibrates the small model's confidence with isotonic regression and reports a 31% cost reduction with two locally served models; Conformal Cascade [3] uses conformal prediction and reports 43%, with formal accuracy guarantees. CARGO [4] and confidence tokens [5] study confidence-aware routing more broadly, and [6] examines when models know what they do not know. These works compare models with models. Here a stage that uses no model at all is part of the comparison.

Language models and supervised classifiers. That a supervised classifier trained on enough labelled data can match or beat a large model prompted without examples is known from the literature on few-shot learning [8, 9]. SetFit [9], for instance, reaches strong accuracy from a few labelled examples per class with small models. The report does not claim this as a finding; it measures where the crossover falls on one workload, with local models and their real costs.

Data and models. The workload is the domain level of CLINC150 [10]; the models are Qwen2.5 [11], served by Ollama; the classifier is the logistic regression of scikit-learn [12] on TF-IDF features. Calibration follows standard practice [13]; the calibration test is that of Hosmer and Lemeshow [14].

3. Setup

3.1 Workload

CLINC150 [10] contains requests to a virtual assistant written by crowd workers, each labelled with one of 150 intents grouped into 10 domains, such as banking, travel or work. We use the domains as departments: each request must be routed to one of 10. The labelled history is the training set, 15,000 requests; the test set has 4,500. Routing a request to one of a closed list of departments has the same structure as triaging a ticket, but the requests are not tickets, and section 6 returns to this.

The 4,500 test requests are split once, with a fixed seed, into a calibration half and a test half of 2,250 each. Every threshold is chosen on the calibration half; every accuracy is measured on the test half, with 95% bootstrap intervals over 2,000 resamples of the test half. The intervals hold the thresholds fixed: they do not include the variability of the split or of the threshold choice. Out-of-scope requests, which CLINC150 also provides, are not used.

3.2 Models

Four open-weight models of the Qwen2.5 family [11], with 0.49, 1.54, 3.09 and 7.61 billion parameters, in the default quantized versions distributed by Ollama, run locally on an office PC (Intel Core Ultra 7 265, 32 GB of memory, no graphics card). Each model sees the list of the 10 departments, each described by the topics it handles (the names of its 15 intents in CLINC150), and no example requests, and answers with the name of one department. Decoding is greedy. An answer that names one of a department's topics instead of the department is counted as that department, and near-misses in spelling are matched to the closest name; answers that match nothing count as wrong. The confidence signal is the probability the model gives to its

answer, the product of the probabilities of its tokens. Cost is the compute time the server reports per request (reading the request and generating the answer; loading the model is excluded), with the instructions cached between requests, as a server in production would run them. A model's cost is its mean over all 4,500 requests, and a cascade's cost is computed from these means.

3.3 Two ways of using the history

Lookup. Each request is compared with every past request by TF-IDF cosine similarity over words and word pairs. If the five closest past requests all carry the same department, and the closest is similar enough, the lookup answers with that department; otherwise it passes the request on. The similarity threshold is chosen on the calibration half, as part of each cascade (section 3.4).

Classifier. A logistic regression on TF-IDF features of words and word pairs (sublinear term frequency, regularisation C = 10, otherwise scikit-learn defaults), trained on the labelled history, with no further tuning. It always answers.

Classifier, then model. The classifier answers when its highest class probability reaches a threshold, otherwise the 7.6B model answers. The threshold (a grid of 0.02, on the raw probability) is the one with the highest accuracy on the calibration half; cost does not enter the choice. Its compute is roughly the share of requests it sends to the model.

3.4 Cascades

Stages run in order; a stage's answer is accepted when its calibrated confidence reaches a threshold, otherwise the request moves on, and the last stage always answers. Confidence is calibrated on the calibration half by quantile bins. The threshold is the one with the lowest cost on the calibration half among those that keep accuracy within half a point of the largest model alone. When a cascade begins with the lookup, its similarity threshold is chosen together with the confidence threshold. Saving is one minus the cascade's cost over the cost of the largest model alone.

4. Results

4.1 The models alone

Table 1: Each model alone, with no example requests. Right answers on the test half; seconds per request averaged over all 4,500 requests.

Model	Right answers	Seconds per request	Separation of right and wrong answers (AUROC)
0.5B	43.1%	0.088	0.73
1.5B	72.4%	0.171	0.71
3.1B	69.9%	0.296	0.74
7.6B	86.6% (85.2% to 88.0%)	0.591	0.77

Accuracy grows with size, with one inversion between 1.5B and 3.1B. The confidence signals are weak: an AUROC of 0.71 to 0.77 means that a model's probability for its own answer separates its right answers from its wrong ones only moderately. The simulation of [15] had assumed signals between about 0.86 and 0.96.

4.2 Cascades of models only cost more

Every cascade made only of models, from two to four stages, costs more than the 7.6B model alone at the same or slightly lower accuracy: from 15% more (0.5B, then 7.6B) to 74% more (all four). With signals this weak, a threshold that keeps accuracy within half a point sends almost every request to the large model anyway, and the small models' compute is added on top. With 1.5B then 7.6B, the small model keeps only 5% of requests; with 0.5B then 7.6B, it keeps none.

This is the opposite of the 31% and 43% savings reported by [2, 3] on other tasks and models. Tasks, models and tolerances differ, but a likely difference is the confidence signal: a cascade saves only if the small model knows when it is right.

4.3 The history alone

At a similarity of 0.3, the value every cascade with the lookup selected on the calibration half (and the lowest value of the grid, so lower ones were not tried), the lookup answers 63% of the test requests and gets 98.7% of those right, in about a third of a millisecond per request. Raising the similarity threshold lowers its coverage (59% at 0.4, 35% at 0.6, 20% at 0.7) while its accuracy stays near 99%.

The classifier answers all of them:

Table 2: The history used by a classifier, against the largest model.

Approach	Right answers	Compute per request
Classifier on 15,000 labelled requests	**95.9%** (95.1% to 96.7%)	about 0.01 ms
7.6B model alone, no examples	86.6% (85.2% to 88.0%)	591 ms

The classifier's time is a batched prediction measured on a different machine from the models', which ran one request at a time; it is only an order of magnitude, tens of thousands of times smaller than the 7.6B model's either way. On this task the labelled history, used by a standard classifier, beats the largest model by more than 9 points.

4.4 History in front of the models

Placed before the models, the lookup changes the economics of every cascade (table 3).

Table 3: Cascades with and without the lookup, on the test half.

Design	Compute saved against 7.6B alone	Accuracy difference, points	Share answered at each stage
lookup, 1.5B, 7.6B	89% (89% to 90%)	+0.5 (-1.2 to +2.1)	63% / 37% / 0%
lookup, 7.6B	63% (61% to 65%)	+5.2 (+4.4 to +6.3)	63% / 37%
1.5B, 7.6B (no history)	-24% (-25% to -23%)	-0.1 (-0.4 to +0.1)	5% / 95%

The lookup followed by the 1.5B model matches the 7.6B model alone at about a ninth of its compute; in this configuration the 7.6B model receives no requests at all. The lookup followed by the 7.6B model is 5 points more accurate than the 7.6B model alone and saves 63% of its compute. Both are dominated, on this task, by the classifier of section 4.3, which is more accurate than either at a negligible cost.

4.5 How much history is needed

The history is not always large. Figure 1 shows what happens when only a few labelled examples per department are available, drawn at random from the 15,000 (five draws per size).

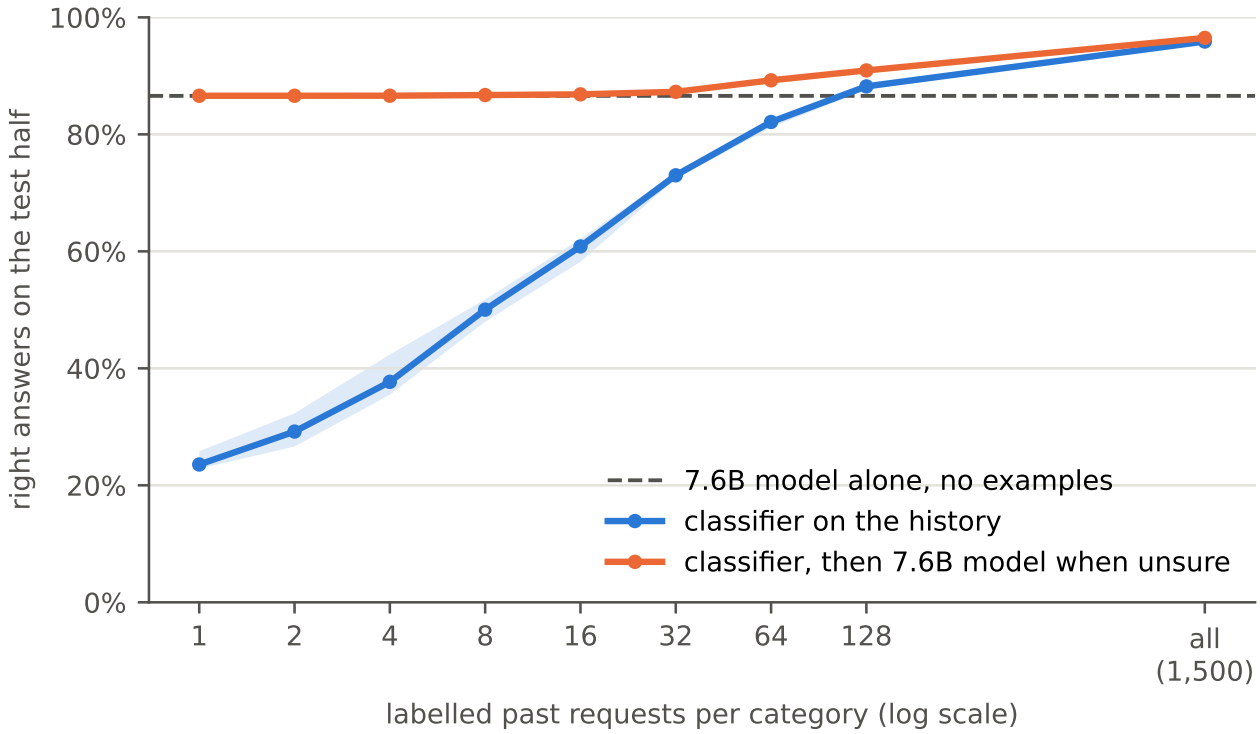


Figure 1: Right answers on the test half as the labelled history grows. Blue: the classifier alone (band: range over five draws). Orange: the classifier, which hands to the 7.6B model the requests on which it is not confident enough, with the threshold chosen on the calibration half. Dashed: the 7.6B model alone, with no examples.

Table 4: The classifier and the classifier-then-model combination with a growing history (mean of five draws; paired bootstrap intervals). The 7.6B model alone gets 86.6%.

Labelled examples per department	Classifier alone	Classifier, then 7.6B	Against 7.6B alone, points	Share sent to 7.6B
8	50.0%	86.7%	+0.1 (+0.0 to +0.3)	94%
32	73.0%	87.3%	+0.7 (+0.1 to +1.3)	57%
64	82.1%	89.2%	+2.7 (+1.9 to +3.4)	39%
128	88.2%	90.9%	+4.4 (+3.3 to +5.3)	22%
1,500 (all)	95.9%	96.5%	+9.9 (+8.5 to +11.3)	2%

Three regimes appear:

- **Little history** (up to about 16 examples per department): the language model carries almost everything. It reads the department descriptions and needs no examples; the classifier cannot.
- **Some history** (about 32 to 128): the combination is the best design. With 64 examples per department it is 2.7 points more accurate than the 7.6B model alone while sending it only 39% of the requests; with 128, 4.4 points while sending it 22%.
- **Ample history** (more than about 100 per department): the classifier alone overtakes the language model. The model still adds something to the combination at 128 examples (+2.7 points over the classifier alone), and little with the full history (+0.6).

4.6 The 5x rule on real models

The simulation of [15] predicted that, with neighbouring models less than 5x apart in cost, a middle model would not pay and a rule stage would. Here every triple of models is less than 5x apart (the average step is 1.8x to 2.6x). Of the four triples, adding a middle model clearly lost in three and clearly gained in one (0.5B, 1.5B, 3.1B, by 48 points); adding the lookup clearly gained in all four, by 89 to 113 points of saving ("clearly": the whole 95% interval on one side of zero). These gains are measured against two-stage cascades that themselves lose money, and the four triples share models, so they are not independent tests. Within its lower range the rule held in three of four cases for the middle model and in all four for the lookup. Its upper range, 5x and beyond, is not tested by these models. The more important correction is to the simulation itself: a real lookup stage covered 63% of requests, not the 10% to 30% assumed, and the real confidence signals were weaker than any simulated one.

4.7 Diagnostics

The models' errors are only weakly correlated (0.12 to 0.31), so escalation can in principle help: the large model often gets right what the small one gets wrong. The difficulty is knowing when to escalate. On the test half the calibrated confidence of the 7.6B model fails the calibration test at the 5% level (chi-square 12.1 with 4 degrees of freedom, $p = 0.016$), though not after correcting for four tests; the other three pass.

5. What this suggests

For an organisation that classifies requests and has, or can build, a history of classified cases, the measurements suggest an order:

1. **Capture the history first.** The labelled past cases were worth more than any model tested. Their value depends on how many there are, how well they are labelled and how current they are.
2. **Use it with the simplest tool that works.** A lookup that answers only when the past agrees is nearly always right on what it answers; a standard classifier answers everything and, with enough history, beats every model tested.
3. **Keep the language model for what the history does not cover**: departments with few past cases, as measured here, and, as a hypothesis this test did not cover, new categories and requests unlike anything seen. There the model's ability to read a description, with no examples, is what the classifier lacks.
4. **Do not count on a cascade of models to save** unless the small model's confidence is shown, on your own data, to separate its right answers from its wrong ones.

The same history also makes the model accountable: with past outcomes, calibration and drift can be measured, and a design can be chosen on numbers rather than on the model's size.

6. Limits

- **One task.** Ten broad departments with 1,500 labelled examples each is an easy case for a classifier. Real ticket queues have more categories, uneven volumes, noisier labels and shifting topics; the crossover of section 4.5 may move in either direction.
- **The requests are not tickets.** They are short requests to an assistant, written by crowd workers, with the structure of routing but not its material. Close paraphrases between the history and the test requests favour the lookup and the classifier.
- **One model family, one machine, no examples in the prompt.** Larger or different models, other quantizations, or prompts with examples taken from the history could change the models' results; a commercial frontier model was not tested.
- **Nothing was tuned.** Neither the classifier nor the models' prompt was optimised. Tuning either could move the gap of section 4.3 in either direction.
- **Costs.** Model costs are compute time on one CPU; the classifier was timed on another machine, in batch, and is given as an order of magnitude. Electricity, hardware and maintenance are not included.
- **Random history.** The small histories of section 4.5 are random samples. A real organisation's history is not random: it over-represents frequent requests and lags behind new ones.

What would count against the conclusion: a realistic workload where a classifier trained on the available history does not beat a language model that sees no examples; or one where a language model given examples from the history beats both the classifier and the combination at comparable cost.

7. Next steps

The protocol is ready for three extensions: the same models with examples from the history in the prompt; a second model family; and a harder workload with many categories and few examples each (Banking77, 77 categories, is already supported by the code). The most useful contribution would be a real one: a history of service tickets, and what each approach achieves on it.

Reproducibility

Code, the stored answer of every model to every request, and this report in English and Italian are at https://github.com/gionnysimonetti-dev/calibrated-funnel under the MIT License, archived on Zenodo. From the stored outputs, `python -m realtest.analyze --task departments --shots 0` writes `realtest/REPORT_departments.md` (costs, AUROC and lookup in sections 4.1 and 4.3; sections 4.2, 4.4, 4.6, 4.7), and `python -m realtest.baselines --task departments` writes `realtest/BASELINES_departments.md` (accuracies with intervals in sections 4.1 and 4.3; section 4.5) and the data of figure 1, which `python paper/make_figure_history.py` draws. Running the models again needs Ollama and about two hours on the PC described in section 3.2: `python -m realtest.run_models --task departments --shots 0`.

Use of AI

The code and part of the analysis were developed with an AI assistant (Claude, Anthropic), which also assisted in drafting this report. The author designed the study, ran the models, reviewed the results and takes responsibility for the content.

References

1. L. Chen, M. Zaharia, J. Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176, 2023.
2. V. Kotte. UCCI: Calibrated Uncertainty for Cost-Optimal LLM Cascade Routing. arXiv:2605.18796, 2026.
3. Y. Dou, S. Lian, S. Li. Conformal Cascade: Distribution-Free Accuracy Guarantees for Multi-Tier LLM Inference. arXiv:2607.25018, 2026.
4. A. Barrak, Y. Fourati, M. Olchawa, E. Ksontini, K. Zoghlami. CARGO: A Framework for Confidence-Aware Routing of Large Language Models. arXiv:2509.14899, 2025.
5. Y.-N. Chuang, P. K. Sarma, P. Gopalan, J. Boccio, S. Bolouki, X. Hu, H. Zhou. Learning to Route LLMs with Confidence Tokens. arXiv:2410.13284, 2024.
6. C. Hao, W. Lu, Y. Ishiwaka, Z. Li, W. Wan, Y. Chen. When Models Know When They Do Not Know: Calibration, Cascading, and Cleaning. arXiv:2601.07965, 2026.
7. P. Aggarwal, A. Madaan, et al. AutoMix: Automatically Mixing Language Models. arXiv:2310.12963, 2023.
8. T. Brown et al. Language Models are Few-Shot Learners. NeurIPS, 2020.
9. L. Tunstall, N. Reimers, U. E. S. Jo, L. Bates, D. Korat, M. Wasserblat, O. Pereg. Efficient Few-Shot Learning Without Prompts. arXiv:2209.11055, 2022.
10. S. Larson et al. An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction. EMNLP-IJCNLP, 2019.
11. Qwen Team. Qwen2.5 Technical Report. arXiv:2412.15115, 2024.
12. F. Pedregosa et al. Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, 2011.
13. C. Guo et al. On Calibration of Modern Neural Networks. ICML, 2017.
14. D. W. Hosmer, S. Lemeshow. Goodness-of-fit tests for the multiple logistic regression model. Communications in Statistics, 1980.
15. G. Simonetti. The 5x Rule: Choosing the Third Stage of a Confidence-Gated Model Cascade. Technical report, draft 0.1, October 2026. In the same repository, paper/five-x-rule.md.